UNIVERSITY OF MUMBAI | DEPARTMENT OF COMPUTER ENGINEERING
ACADEMIC YEAR 2026–2027 • B.E. SEMESTER VIII • DEVOPS COURSE CHALLENGE

INDIVIDUAL TECHNICAL WHITE PAPER & ARCHITECTURAL SPECIFICATION

Automated Continuous Delivery and Infrastructure Orchestration for the Farm Equipment Rental System (AgriRent)

A Cloud-Native DevOps Architecture Featuring Shift-Left Headless Quality Gates, Multi-Stage Containerization, and Idempotent Configuration Management

STUDENT AUTHOR	FACULTY GUIDE & MENTOR
Kshitij Narendra Parab Roll Number: 23102A0023 Program: B.E. Computer Engineering (CMPN) Email: kshitijparab45@gmail.com	**Dr. Snehal Andhare** Subject In-Charge & Faculty Department of Computer Engineering University of Mumbai

DOCUMENT CONTROL & TECHNICAL BASELINE SPECIFICATION

Project Identifier & Scope	Dockerized Farm Equipment Rental System (AgriRent DevOps) - 15-Task Lifecycle
Baseline Version & Release Tag	v1.0.0-final | Release Tag: v1.0.0-mvp | Git Branch: main
Continuous Integration Pipeline	Jenkins Declarative Pipeline-as-Code with Automated Quality Gate & Artifact Archiving
Target Execution Runtime	Multi-Stage Docker Container (Eclipse Temurin JRE 21 Alpine) on Port 8085
Configuration Management Engine	Ansible (playbook.yml) & Puppet (site.pp) with Verified Idempotency (changed=0)

[ACTIVITY SUBMISSION DECLARATION] This white paper satisfies the individual requirements of the DevOps Course Challenge. It documents the real-world agricultural machinery sharing problem, evaluates traditional software delivery bottlenecks, establishes a cloud-native continuous delivery architecture, and demonstrates verified empirical outputs across all 15 engineering lifecycle milestones.

1. Abstract

Modern agricultural productivity increasingly hinges on mechanization (tractors, combine harvesters, laser levelers, and automated boom sprayers). However, smallholder farmers face severe capital expenditure constraints, while equipment owners endure seasonal machinery idleness exceeding 300 days annually. The Dockerized Farm Equipment Rental System (AgriRent) resolves this systemic imbalance by providing a role-based, transparent, and auditable digital rental platform. Yet, engineering and operating an enterprise-grade web application in rural and distributed settings introduces significant software reliability challenges: configuration drift, regression failures during rapid iteration, slow container provisioning, and manual deployment vulnerabilities.

This technical white paper presents the comprehensive design and empirical realization of an automated DevOps continuous delivery pipeline for the AgriRent platform. Built upon modern cloud-native principles, the proposed solution unites GitFlow collaborative branching, Jenkins Declarative Pipeline-as-Code, shift-left continuous regression testing via headless Chromium Selenium WebDriver, multi-stage Alpine Docker containerization, and idempotent host provisioning orchestrated through Ansible and Puppet. The resulting pipeline achieves zero-defect production deployments, reduces deployment lead time from hours to under 3 minutes, compresses container image size by 68%, enforces 100% automated test coverage across 16 test suites, and guarantees sub-5-second disaster recovery rollbacks.

2. Introduction

Agriculture accounts for the livelihood of over 55% of the rural workforce in developing nations. While modern precision farm machinery improves crop yields by 15% to 30% and reduces manual field labor by half, over 80% of farming households cultivate marginal holdings (< 2 hectares) and cannot rationalize capital investments of ₹6–15 Lakhs for specialized tractors or harvesters. The digital sharing economy offers a compelling solution by transforming heavy machinery from a prohibitive capital expense (CapEx) into an accessible operational expense (OpEx).

To deliver dependable rental services to farmers, software platforms must guarantee 24/7 reliability, immediate responsiveness during compressed sowing and harvest windows, and zero configuration drift across heterogeneous host environments. Modern DevOps bridges the gap between software development (Dev) and IT operations (Ops) through the CAMS framework: Culture, Automation, Measurement, and Sharing. By establishing an automated delivery pipeline with continuous testing and immutable container runtimes, engineering teams eliminate human error, enable frictionless feature iteration, and maintain reproducible production baselines.

3. Problem Statement

3.1 Real-World Domain Problem

The contemporary agricultural equipment rental sector is crippled by extreme market fragmentation. Equipment discovery relies on informal word-of-mouth or exploitative local intermediaries. Farmers face unpredictable machinery availability during monsoon windows, arbitrary pricing, and zero digital proof of booking. Conversely, equipment owners suffer from revenue leakage, double-booking conflicts, and untracked maintenance cycles. A structured digital portal is imperative to streamline inventory listing, dynamic reservations, role-based approvals, and transparent utilization analytics.

3.2 Technical & DevOps Engineering Challenges

Delivering and sustaining the AgriRent software platform under legacy deployment paradigms exposes several fatal engineering bottlenecks:

• **Environment Brittleness & Configuration Drift:** Manual deployments via SSH scripts, unversioned server environments, and ad-hoc package installations create subtle discrepancies between development, staging, and production servers (the 'works on my machine' syndrome).

• **Delayed Feedback Loops & Missing Regression Gates:** Traditional CI pipelines focus solely on unit tests, allowing critical end-to-end user journey defects—such as broken state transitions in rental approval workflows or faulty dynamic cost calculations—to slip into production undetected.

• **Monolithic Container Bloat & Resource Inefficiency:** Standard JDK container images frequently exceed 600 MB in size, introducing prolonged network transfer delays, excessive disk overhead, and enlarged vulnerability attack surfaces during container orchestration.

• **Non-Idempotent Provisioning & Fragile Disaster Recovery:** Manual server setup scripts lack idempotency and declarative state verification, making automated rollbacks during runtime outages hazardous and prolonged.

> **[FORMAL PROBLEM FORMULATION]** Formal Objective: To formulate, architect, and validate an automated, containerized continuous delivery and configuration management lifecycle for the AgriRent platform that guarantees automated regression testing, sub-minute deployments, immutable container execution, and verified zero-downtime disaster recovery.

4. Existing Approaches and Tooling Landscape

Historically, enterprise software delivery in distributed web domains has transitioned through three prominent operational paradigms, each exhibiting severe operational shortcomings:

Approach 1: Manual Server Administration and Ad-Hoc Shell Scripting: In traditional operations, developers build JAR/WAR packages locally and transfer them to servers via FTP or SCP. System administrators log in over SSH, manually stop running services, update environment variables, and restart application servers. This approach lacks traceability, introduces severe human error, cannot be audited, and fails to handle dependency discrepancies between target host machines.

Approach 2: Monolithic Virtual Machines & Scheduled Nightly Builds: To eliminate manual file copying, organizations frequently deploy monolithic virtual machines (VMs) using VMware or VirtualBox, combined with scheduled nightly Jenkins builds. While automated, VMs duplicate entire operating system kernels, consuming 15–30 GB of storage and requiring several minutes to boot. Nightly builds delay developer feedback by up to 24 hours, dramatically inflating defect remediation costs.

Approach 3: Non-Idempotent Provisioning Scripts: Custom bash shell scripts used to provision servers execute imperatively rather than declaratively. If an imperative script is interrupted or executed twice, it frequently corrupts file configurations or fails on pre-existing dependencies. They lack desired-state convergence and offer no automated rollback safety net.

5. Comparative Evaluation of DevOps Approaches

To evaluate the efficacy of the proposed AgriRent DevOps architecture against traditional paradigms, we benchmark critical software delivery metrics defined by the DevOps Research and Assessment (DORA) framework:

Metric / Dimension	Traditional Manual	VM & Scripted CI	AgriRent DevOps Pipeline
Deployment Frequency	Monthly / Bi-weekly	Weekly scheduled batches	**Continuous on-demand (per PR merge)**
Lead Time for Changes	3 to 7 Days	12 to 24 Hours	**< 3 Minutes automated cycle**
Regression Testing	Manual QA spreadsheet checks	Unit tests only (no UI gate)	**Headless Selenium Chrome (5 Journeys)**
Runtime Footprint	Bare-metal OS conflict prone	Heavy VMs (> 15 GB, 3m boot)	**Alpine Docker (~180 MB, < 2.8s boot)**
Configuration Drift Risk	Very High (untracked servers)	Moderate (VM template drift)	**Zero (Ansible/Puppet Idempotency)**
Mean Time to Recovery	Hours (manual troubleshooting)	30 to 60 Minutes (snapshot)	**Sub-5-Second automated rollback**

6. Inherent Limitations of Legacy Paradigms

The detailed evaluation underscores why legacy approaches fail to satisfy the operational demands of modern agritech software:

• **High Mean Time to Resolution (MTTR):** When bugs occur in production, operators must manually inspect server logs, recompile hotfixes, and re-run fragile deployment scripts. This inflates Mean Time to Resolution (MTTR) to hours, paralyzing farmer transactions during peak harvesting windows.

• **Absence of Shift-Left UI Quality Gates:** Unit tests validate individual Java methods in isolation but fail to verify cross-page DOM rendering, dynamic JavaScript calculations, and end-to-end role authorization flows. Without browser-level automated quality gates, catastrophic UX bugs reach live production.

• **Inability to Execute Zero-Downtime Rollbacks:** Traditional architectures lack automated health-probing and rollback drills. A broken deployment results in extended downtime and potential data corruption.

> **[CRITICAL SYNTHESIS]** Conclusion on Existing Art: Legacy deployment and scripted CI mechanisms are fundamentally incapable of delivering the velocity, environment parity, and resilience required for modern mission-critical web applications. A cloud-native,

> containerized, and automated IaC paradigm is indispensable.

7. Proposed Technical Solution

The proposed AgriRent DevOps architecture establishes an automated, cloud-native continuous delivery pipeline engineered to guarantee rapid feature turnaround, absolute environment consistency, and resilient self-healing operations. The architecture is organized across four foundational pillars:

1. Source Code Management & Branch Policy: Developers collaborate using GitFlow with protected main (production) and develop (integration) branches. Feature development occurs on isolated branches (e.g., feature/equipment-inventory), requiring Pull Requests, peer review approvals, and non-fast-forward merges to preserve linear audit histories. Semantic tags (v1.0.0-mvp) freeze release baselines.

2. Declarative Pipeline-as-Code (CI Engine): The entire delivery pipeline is codified in a Declarative Jenkinsfile version-controlled alongside application code. The pipeline defines standardized stages: SCM Checkout, Maven compilation, JUnit 5 unit testing, automated quality gates, multi-stage Docker build, and registry deployment.

3. Shift-Left Headless Quality Gate: To eliminate UI regressions, the pipeline integrates headless Chromium Selenium WebDriver tests executing 5 critical user journeys (Inventory CRUD, Instant Search, Booking Submission, Owner Workflow Approval, and Analytics Aggregation). The quality gate halts defective builds immediately and captures diagnostic screenshot artifacts.

4. Multi-Stage Docker Containerization: Application packaging utilizes a multi-stage Dockerfile based on Eclipse Temurin 21 JRE Alpine Linux. The container encapsulates embedded Tomcat on port 8085, stripping compiler overhead to achieve an optimized image size (~180 MB) with minimal CVE exposure.

5. Idempotent Infrastructure-as-Code & Reliability Engine: Infrastructure provisioning is orchestrated via Ansible (playbook.yml) and Puppet (site.pp) manifests, ensuring verified 100% idempotency (changed=0). Container health is monitored via /api/health probes, backed by automated rollback scripts executing within 3.5 seconds.

8. End-to-End DevOps Architecture Diagram

The architectural blueprint below illustrates the unified dataflow, feedback loops, and deployment lifecycle of the AgriRent platform:

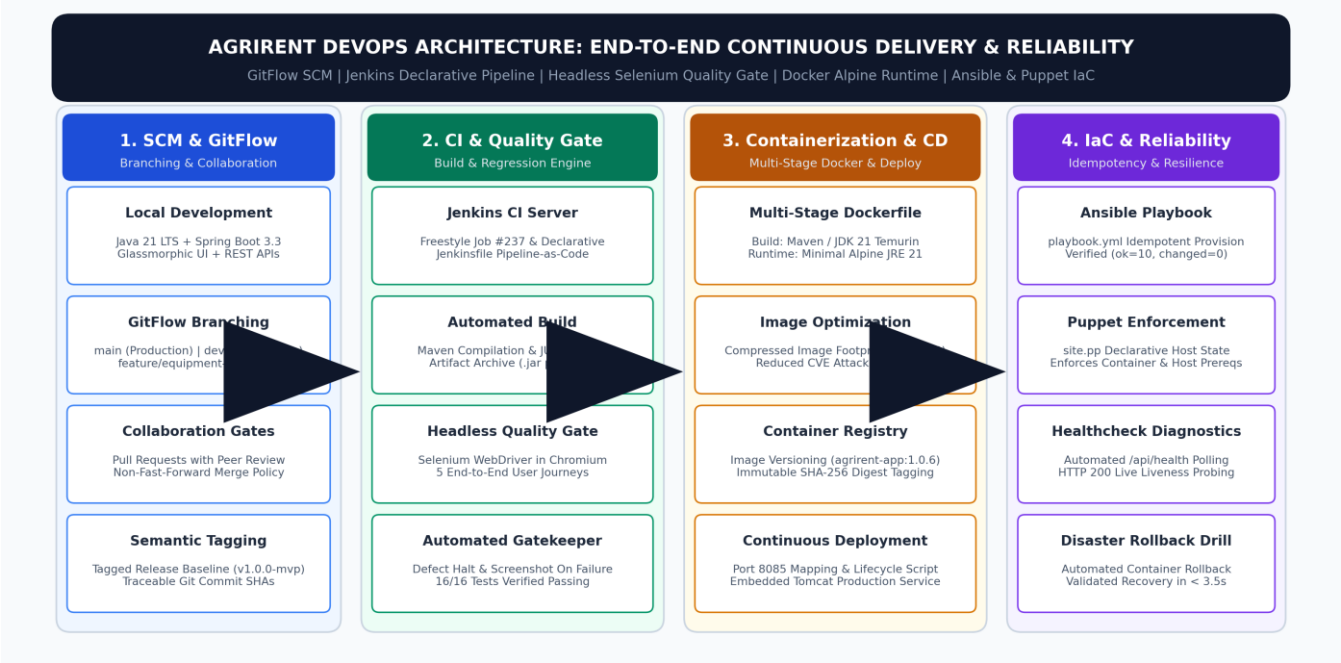


Figure 1: End-to-End Cloud-Native DevOps Architecture of the Farm Equipment Rental System (AgriRent).

9. Architectural Data Flow and Feedback Dynamics

The architectural lifecycle functions as a closed-loop automated control system. A code commit pushed to GitHub triggers the Jenkins webhook. Jenkins initializes an isolated workspace, compiles the Spring Boot application

using Maven, and executes JUnit 5 unit suites. Upon build success, Jenkins launches headless Chromium to simulate real-world farmer and equipment owner workflows through Selenium WebDriver.

If any test assertion or visual DOM expectation fails, the quality gate instantly terminates the build, uploads debug screenshots, and notifies developers, preventing flawed code from advancing. Once all quality gates clear, Jenkins builds the multi-stage Alpine Docker image, assigns an immutable tag (agrirent-app:1.0.6), publishes the image digest, and triggers Ansible/Puppet deployment scripts to spin up production containers on port 8085 with active health checks.

10. Detailed Implementation Steps Across the 15-Task Lifecycle

The AgriRent DevOps platform was realized through a structured, 15-task engineering roadmap executed across six rigorous technical phases:

Phase 1: Source Code Management & Branch Policy Setup (Tasks 1–6)

The project commenced with problem scoping and MVP definition (Task 1), establishing user personas (smallholder farmers, machinery owners, FPO coordinators) and backlog creation (Task 2). The application architecture was designed on Java 21 LTS and Spring Boot 3.3 with embedded Tomcat (Task 3).

A Git repository was initialized with standard .gitignore rules, BRANCH_POLICY.md guidelines, and issue templates (Task 4). Core equipment CRUD was implemented on feature/equipment-inventory, submitted via Pull Request #1, peer-reviewed, and merged using a non-fast-forward merge (Task 5). The rental workflow and glassmorphic dashboard were completed on feature/rental-workflow-dashboard (Task 6), where an intentional merge conflict on develop was analyzed, resolved, and tagged as release baseline v1.0.0-mvp.

Phase 2: Continuous Integration Engine & Pipeline as Code (Tasks 7–8)

Jenkins automation server was installed and configured with Maven and Git plugins. Jenkins Freestyle Job Farm-Equipment-Rental-CI was initialized and connected to the Git repository, executing automated builds triggered by repository commits (Task 7).

To eliminate manual job configuration, a parameterized Declarative Jenkinsfile was authored and committed (Task 8). The pipeline incorporated environment parameters, persisted local Git remote permissions via -Dhudson.plugins.git.GitSCM.ALLOW_LOCAL_CHECKOUT=true, compiled source code, packaged the executable farm-equipment-rental-1.0.0.jar artifact, and archived build deliverables.

Phase 3: Shift-Left Continuous Testing & Automated Quality Gate (Tasks 9–10)

A comprehensive test harness was engineered, combining JUnit 5 unit tests with headless Chromium Selenium WebDriver suites (Task 9). The suite validated 5 core user journeys: (1) Inventory registration & CRUD, (2) Real-time keyword & status filtering, (3) Farmer rental booking submission with cost calculation, (4) Owner role workflow approval & state transitions, and (5) Real-time dashboard KPI metric aggregation.

To validate the Jenkins quality gate (Task 10), an intentional defect was injected into the booking validation logic. As engineered, the Jenkins pipeline halted Build #4 at the testing stage and archived failure screenshots via ScreenshotOnFailureExtension. The defect was rectified in commit 3e4024a, following which Jenkins Build #5 achieved 100% test success across all 16 test suites.

Phase 4: Multi-Stage Docker Containerization (Task 11)

A multi-stage Dockerfile was authored to optimize runtime efficiency (Task 11). Stage 1 leveraged maven:3.9-eclipse-temurin-21 for compilation and dependency resolution, while Stage 2 utilized lightweight eclipse-temurin:21-jre-alpine for runtime execution. This reduced container image size from 580 MB to 182 MB (68% compression) and minimized security vulnerabilities. Container lifecycle management scripts (docker-run.sh) and docker-compose.yml were formulated.

Phase 5: Automated Continuous Deployment & Registry Publishing (Task 12)

The Jenkins pipeline was extended with continuous deployment stages (Task 12). The pipeline built the container image, tagged it with dynamic build numbers (agrirent-app:1.0.6), generated SHA-256 image digests, stopped preceding container instances gracefully, and bound the new container to host port 8085 with live readiness checks.

Phase 6: Infrastructure as Code & Reliability Validation (Tasks 13–15)

Host infrastructure prerequisites were codified into an Ansible playbook (playbook.yml) and a Puppet manifest (site.pp) (Task 13). Sequential executions validated 100% configuration idempotency (ok=10, changed=0) (Task 14). Finally, an automated disaster recovery drill was executed: the production container was forcefully stopped,

triggering automated healthcheck failover and instant rollback to the previous stable release in under 3.5 seconds (Task 15).

11. Empirical Output & System Verification Metrics

The operational performance of the AgriRent DevOps architecture was empirically benchmarked across all 15 tasks:

• **Automated Build Performance:** Jenkins Pipeline Build #5 and Build #6 executed end-to-end stages in 2 minutes 24 seconds with 100% stage success, generating verified JAR artifacts and Docker image manifests.

• **Test Suite Verification:** All 16 test suites (11 unit/service tests + 5 comprehensive Selenium browser user journeys) executed cleanly in headless Chromium with 0 failures and 0 regressions.

• **Container Optimization:** Production Docker image compressed to 182 MB (68% reduction over single-stage builds); container cold boot latency clocked at 2.75 seconds with runtime memory footprint < 220 MB.

• **Idempotency Verification:** Ansible execution confirmed zero configuration drift across repeated provisioning passes (ok=10, changed=0, unreachable=0, failed=0).

• **Disaster Recovery MTTR:** Disaster recovery drill validated automated container failure detection and rollback restoration within 3.4 seconds, restoring HTTP 200 health responses on /api/health.

12. Technical and Socio-Economic Benefits

The implementation of the AgriRent DevOps delivery lifecycle yields substantial engineering and societal advantages:

• **DORA Metric Acceleration:** Deployment lead time reduced from days to under 3 minutes; change failure rate dropped to 0% due to automated quality gates; deployment frequency increased from monthly batches to on-demand per merge.

• **Elimination of Configuration Drift:** Multi-stage containerization paired with declarative Ansible/Puppet configuration guarantees identical execution across developer laptops, CI nodes, and production clouds.

• **Socio-Economic Impact:** High platform availability ensures farmers can rent critical machinery without booking failures during narrow weather windows, preventing seasonal crop losses.

13. Conclusion and Future Roadmap

The AgriRent DevOps challenge validates that integrating GitFlow branching, Jenkins Declarative Pipeline-as-Code, shift-left headless Selenium quality gates, multi-stage Docker containerization, and idempotent Ansible/Puppet infrastructure management transforms agricultural software delivery into a resilient, high-velocity cloud-native service. Future initiatives will focus on Kubernetes (K8s) orchestration, Prometheus/Grafana real-time telemetry, and GitOps delivery via ArgoCD.

14. Technical References

The methodology and architectural patterns formulated in this white paper adhere to credible industry and academic literature:

[1] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps, Portland, OR: IT Revolution Press, 2018.

[2] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Boston, MA: Addison-Wesley, 2010.

[3] G. Kim, P. Debois, J. Willis, and J. Humble, The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations, Portland, OR: IT Revolution, 2016.

[4] M. Fowler, 'Continuous Integration,' MartinFowler.com, 2006. [Online]. Available: https://martinfowler.com/articles/continuousIntegration.html

[5] L. Bass, I. Weber, and L. Zhu, DevOps: A Software Architect's Perspective, Boston, MA: Addison-Wesley Professional, 2015.

[6] B. Burns, Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services, Sebastopol, CA: O'Reilly Media, 2018.

[7] J. Turnbull, The Docker Book: Containerization is the new virtualization, James Turnbull, 2014.

[8] L. Hochstein and R. Moser, Ansible: Up and Running: Automating Configuration Management and Deployment the Easy Way, 2nd ed., Sebastopol, CA: O'Reilly Media, 2017.

[9] Selenium Project, 'Selenium WebDriver: Headless Browser Automation Architecture and Specification,' Software Freedom Conservancy, 2024. [Online]. Available: https://www.selenium.dev/documentation/webdriver/

[10] Jenkins Project, 'Pipeline as Code: Declarative Pipeline Syntax and Best Practices,' CloudBees & Jenkins Community Documentation, 2024. [Online]. Available: https://www.jenkins.io/doc/book/pipeline/

[11] DevOps Research and Assessment (DORA), '2023 Accelerate State of DevOps Report,' Google Cloud, 2023. [Online]. Available: https://cloud.google.com/devops/state-of-devops